

Artificial Intelligence

Search

Where We Are

In the previous lecture we studied **Intelligent Agents**.

Simple Reflex



Model-Based



Goal-Based



Search

Observation

A goal-based agent often does not know immediately which action should be taken. Instead, it must search for a sequence of actions that achieves its goal.

Part I: Problem Solving as Search

Why Search?

Many AI systems must solve problems for which the solution is not immediately obvious.

Examples include

- planning a route through a city;
- solving a puzzle;
- playing a game of chess;
- navigating a robot through a warehouse;
- proving a mathematical theorem.

In each case, the agent must explore a space of possible solutions before deciding what to do.

Search as Problem Solving

Informal Definition

Search is the computational process of exploring a space of possible solutions in order to find one that satisfies a specified goal.

Every search problem has

- an initial state;
- a set of possible actions;
- a collection of reachable states;
- one or more goal states.

The objective is to discover a sequence of actions leading from the initial state to a goal state.

Formal Definition of a Search Problem

A search problem may be represented by the tuple

$$\mathcal{P} = (S, A, T, s_0, G, c),$$

where

Symbol	Meaning
S	Set of states
A	Set of actions
T	Transition function
s_0	Initial state
G	Goal state(s)
c	Cost function (optional)

This mathematical representation is independent of any particular search algorithm.

Running Example: Warehouse Navigation

Consider the warehouse introduced in the laboratory.

```
#####  
#S...#.....G#  
#..#...#####..#  
#...##.....#  
#.#####.###.#.###..#  
#.....#.....#  
#####
```

The warehouse vehicle must determine a collision-free path from the starting position S to the goal G.

Summary

In this section, we learnt that a search problem may be represented by

$$\mathcal{P} = (S, A, T, s_0, G, c),$$

where

- S is the set of states;
- A is the set of actions;
- T is the transition function;
- s_0 is the initial state;
- G is the set of goal states;
- c is an optional cost function.

Question:

Given a search problem, *which state should we explore next?*

Part II: Classical Search

Blind versus Informed Search

Search algorithms may be divided into two broad classes.

Blind Search	Informed Search
No heuristic information	Uses heuristic information
Explores systematically	Uses domain knowledge
No estimate of goal direction	Estimates distance to goal

We begin with **blind search**

Breadth-First Search

Key Idea

Expand the **shallowest unexplored node** first.

BFS explores the search tree one level at a time.

Typical expansion order

S

A

B

C

D E F G H I

Expansion sequence

$$S \rightarrow A, B, C \rightarrow D, E, F, G, H, I$$

Breadth-First Search Algorithm

BFS maintains a queue containing the frontier.

Put the start node into the queue.

Repeat

- Remove the first node.

- If it is the goal, stop.

- Otherwise

 - Expand the node.

 - Add all unexplored children
to the end of the queue.

Until the queue is empty.

Data structure: Queue (First-In First-Out).

Properties of Breadth-First Search

Property	BFS
Complete	Yes
Optimal	Yes (equal step costs)
Time	$O(b^d)$
Space	$O(b^d)$

where

- b is the branching factor;
- d is the depth of the shallowest solution.

Depth-First Search

Key Idea

Expand the **deepest unexplored node** first.

Rather than exploring level by level, DFS follows one branch as deeply as possible before backtracking.

Typical exploration

$$S \rightarrow A \rightarrow D \rightarrow \dots$$

before returning to explore the remaining branches.

Depth-First Search Algorithm

DFS maintains a stack containing the frontier.

Push the start node onto the stack.

Repeat

 Pop the top node.

 If it is the goal, stop.

 Otherwise

 Expand the node.

 Push unexplored children
 onto the stack.

Until the stack is empty.

Data structure: Stack (Last-In First-Out).

Properties of Depth-First Search

Property	DFS
Complete	Not always
Optimal	No
Time	$O(b^m)$
Space	$O(bm)$

where

- b is the branching factor;
- m is the maximum search depth.

DFS generally requires much less memory than BFS but may explore very deep branches before finding a solution.

Breadth-First Search versus Depth-First Search

Property	BFS	DFS
Frontier data structure	Queue	Stack
Search strategy	Level by level	One branch at a time
Complete	Yes	Not always
Optimal	Yes (equal costs)	No
Memory usage	High	Low
Typical application	Shortest paths	Deep exploration

Neither algorithm makes use of any knowledge about where the goal is located.

The Limitation of Blind Search

Suppose we are searching for a destination on a map.

Should we

- explore every possible direction equally, or
- use our knowledge of the destination to guide the search?

Blind search ignores all information about the location of the goal.

Can we search more intelligently?

Informal Definition

A **heuristic** is a function that estimates how close a state is to a goal.

Mathematically,

$h(n)$ = estimated cost from node n to the goal.

Examples include

- straight-line distance;
- Manhattan distance;
- estimated travel time.

Example Heuristics

Suppose the goal is located at the lower-right corner of a grid.

Possible heuristics include

- Euclidean distance,

$$h(n) = \sqrt{(x - x_G)^2 + (y - y_G)^2};$$

- Manhattan distance,

$$h(n) = |x - x_G| + |y - y_G|.$$

A heuristic estimates how promising a state appears to be.

Greedy Best-First Search

Key Idea

Always expand the node that appears closest to the goal.

The decision rule is

$$n^* = \arg \min_{n \in \text{OPEN}} h(n).$$

The algorithm always chooses the node with the smallest heuristic value.

Advantages and Limitations of Greedy Search

Advantages

- often explores far fewer nodes than BFS;
- frequently finds a solution quickly;
- simple to implement.

Limitations

- may become trapped following an apparently good path;
- may overlook shorter solutions;
- is not guaranteed to find the optimal solution.

A Better Idea

Greedy search asks only

“How close am I to the goal?”

However, another question is equally important.

“How much has it already cost to reach this node?”

An intelligent search algorithm should consider

- the cost already incurred;
- the estimated cost remaining.

The A* Evaluation Function

A* combines two quantities.

$$f(n) = g(n) + h(n),$$

where

- $g(n)$ is the cost from the start node to n ;
- $h(n)$ estimates the remaining cost to the goal.

Thus,

$$f(n) = \text{cost so far} + \text{estimated cost remaining}$$

The node with the smallest value of $f(n)$ is expanded next.

When is A^* Optimal?

Suppose

$$h(n) \leq h^*(n),$$

where

$$h^*(n)$$

denotes the true remaining cost to the goal.

Such a heuristic is called

admissible.

Theorem

If the heuristic is admissible, A^* is guaranteed to find an optimal solution.

Comparison of Classical Search Algorithms

Algorithm	Uses Heuristic	Complete	Optimal	Memory
Breadth-First	No	Yes	Yes	High
Depth-First	No	No	No	Low
Greedy	Yes	Usually	No	Moderate
A*	Yes	Yes	Yes	Moderate–High

Summary

In this section we introduced four important search strategies.

Breadth-First Search



Depth-First Search



Greedy Search



A*

The key difference between these algorithms is

Which node should be expanded next?

In the next section we shall study search in **continuous spaces**, leading to gradient descent and stochastic gradient descent.

Part III: Continuous Search

From Discrete Search to Continuous Search

So far we have considered search in *discrete* state spaces.

Examples include

- route planning;
- robot navigation;
- game playing;
- puzzle solving.

Question:

What happens if the search space is *continuous* rather than discrete?

Discrete versus Continuous Search

	Discrete Search	Continuous Search
Search Space	States	Parameters
Candidate Solution	Node	Vector
Representation	Graph	$\mathbb{R}^n$
Decision	Expand node	Update parameters
Examples	BFS, DFS, A*	Gradient Descent

The underlying idea is unchanged:

Search for a better solution.

An Optimisation Problem

Suppose every candidate solution is represented by a vector

$$\theta = (\theta_1, \theta_2, \dots, \theta_n).$$

We wish to minimise an objective function

$$L(\theta).$$

Examples include

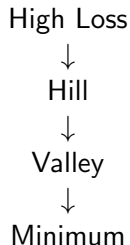
- prediction error;
- manufacturing cost;
- travel time;
- energy consumption.

The objective is

$$\theta^* = \arg \min_{\theta} L(\theta).$$

The Optimisation Landscape

Rather than searching a graph, imagine searching a landscape.



Each point corresponds to one possible solution.

The height represents the value of the objective function.

Our goal is to move towards the lowest point.

Which Direction Should We Move?

In discrete search

Which node should we expand next?

In continuous search

Which direction should we move?

The answer is provided by the **gradient**.

The Gradient

The gradient of a function is

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_n} \right).$$

It points in the direction of

greatest increase

of the objective function.

Therefore

$$-\nabla L(\theta)$$

points towards decreasing values.

Gradient Descent

The basic update rule is

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where

- η is the learning rate;
- $\nabla L(\theta_t)$ is the gradient.

At every iteration we move a small distance in the direction of steepest descent.

Interpreting Gradient Descent

Gradient descent follows a simple cycle.

- ① Evaluate the current solution.
- ② Compute the gradient.
- ③ Move in the downhill direction.
- ④ Repeat.

This is precisely the same philosophy as heuristic search:

Use local information to move towards a better solution.

Choosing the Learning Rate

The parameter η is called the **learning rate**.

Choosing an appropriate value is important.

Learning Rate	Behaviour
Too small	Very slow convergence
Too large	May overshoot the minimum
Appropriate	Steady convergence

The learning rate determines the size of each search step.

Local and Global Minima

Many optimisation problems have more than one minimum.

- A **global minimum** is the best possible solution.
- A **local minimum** is better than nearby solutions, but may not be the best overall.

Gradient descent is a local optimisation method.
Its final solution may depend upon the starting point.

The Cost of Computing the Gradient

Suppose our objective function is

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta),$$

where

- N is the number of training examples;
- $\ell_i(\theta)$ is the loss for example i .

To compute the gradient exactly we must evaluate

$$\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta).$$

For very large datasets this may be computationally expensive.

Stochastic Gradient Descent

Instead of using the entire dataset, we estimate the gradient using a single randomly selected example.

The update becomes

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_i(\theta_t)$$

where i is chosen randomly.

This is called

Stochastic Gradient Descent (SGD).

Mini-Batch Gradient Descent

In practice, we will probably use **mini-batches**.

Instead of

- one example, or
- the entire dataset,

we use a small subset of the data.

Typical batch sizes include

16, 32, 64, 128, 256.

Mini-batch methods provide a good compromise between efficiency and stability.

Gradient Descent versus Stochastic Gradient Descent

Property	Gradient Descent	SGD
Uses all data	Yes	No
Gradient estimate	Exact	Approximate
Speed	Slower	Faster
Noise	Low	Higher
Memory	Higher	Lower
Common in Deep Learning	Rarely	Yes

Search as a Unifying Idea

This module has considered two different kinds of search.

	Discrete Search	Continuous Search
Search Space	States	Parameters
Representation	Graph	$\mathbb{R}^n$
Decision	Expand node	Update parameters
Guidance	Heuristic	Gradient
Examples	BFS, DFS, Greedy, A*	GD, SGD

Although the algorithms differ, they share the same objective:

Search efficiently for a better solution.

Summary

In this module we studied

- search as problem solving;
- blind search;
- heuristic search;
- continuous optimisation;
- gradient descent;
- stochastic gradient descent.

Key Message

Search is a fundamental computational principle in Artificial Intelligence.

Different search algorithms exploit different kinds of information, but all seek the same objective: efficiently discovering high-quality solutions.

Next Lecture:

Knowledge Representation and Logical Reasoning